FlightOPU: An FPGA Overlay Processor for LLM with HBM-Aware Multi-Die Architecture

Chen Wu[1], Shaoqiang Lu[1, 2], Yangbo Wei[1, 2], Junhong Qian[3], Jinlong Yan[1],
Zhanfei Chen[1], Rumin Zhang[1], Xiao Shi[3], and Lei He[1]
[1]*Ningbo Institute of Digital Twin, Eastern Institute of Technology, Ningbo, China*
[2]*Shanghai Jiao Tong University, Shanghai, China* [3]*Southeast University, Nanjing, China*
cwu@idt.eitech.edu.cn

2025 International Conference on Field Programmable Technology (ICFPT) | 979-8-3315-5752-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICFPT67023.2025.00048

***Abstract*—Large language models (LLMs) demand massive computation and memory, which is difficult in edge scenarios. In modern FPGAs with HBM and multi-Die, we observe three challenges: (1) autoregressive LLMs exhibit strong bandwidth sensitivity, yet bandwidth-centric memory–compute balancing remains underexplored; (2) multi-die placement and routing constraints hinder scaling a single large core, limiting flexible expansion and clock frequency; (3) diverse data precisions make fixed-structure PE arrays inefficient, causing idle cycles. To address these issues, we propose FlightOPU, an FPGA overlay processor that establishes a strong memory–compute affinity to accelerate LLM inference. Specifically, it comprises three parts: first, compute resources are allocated per HBM channel, with a processing core per channel, and a nonlinear module and a Sync Router placed on each die to handle multi-core communication. Second, the compiler maps the model to a scalable multi-core system, where inter-core partitioning is column-based and uses interleaved-access scheduling to reduce communication, while cores internally pipeline to overlap computation and load latency. Third, a reconfigurable PE pulsing array can match different bitwidth precisions (e.g., $16 \times 16 \leftrightarrow 8 \times 32$, BF16/FP8/INT4) and maintain input/output bitwidth consistency. Experiments on a V80 FPGA across LLaMA-7B, GPT-2, Qwen-7B, and DeepSeek-7B show up to $2.7\times$ throughput speedup and $5.6\times$ energy-efficiency gains, while maintaining high effective HBM utilization and near-linear scaling with tile count.**

***Index Terms*—FPGA, AI accelerator, LLM, HBM, reconfigurable computing, OPU**

I. Introduction

The recent emergence of LLMs has revolutionized numerous AI applications—including autonomous agents, program synthesis, and multimodal document understanding—by pushing the frontier of neural computation to unprecedented scales. However, deploying such models on *edge or near-edge* platforms introduces stringent constraints on latency, energy consumption, and on-chip memory utilization, which conventional CPU/GPU architectures often fail to meet. These limitations have motivated a surge of research into *FPGA-based accelerators* for large-scale vision and language models, combining algorithm–architecture co-design, domain-specific dataflows, and precision-adaptive computation.

Over the past few years, the FPGA community has made substantial progress across multiple fronts. For **vision Transformers**, accelerator designs targeting Swin and ViT models adopt hierarchical tiling, attention-centric dataflows, and bank-aware memory organizations, achieving notable gains on Xilinx Alveo and U-series devices [1]. Flexible frameworks such as **ProTEA** support runtime-programmable tiling for encoder workloads, while mixed-precision engines like **TATAA** employ reconfigurable arithmetic units to balance accuracy and efficiency [3], [4]. Architectural innovations—including linear and hybrid attention schemes—further align computation with FPGA parallelism and HBM bandwidth [5], [6]. Beyond single devices, **multi-FPGA** studies explore graph-partitioned Transformer mappings, low-latency decoding, and inter-device communication co-optimization [7], [8], [11]. Co-design frameworks integrating quantization, pruning, and hardware adaptation [2], [10], along with recent extensions to diffusion and multimodal models [9], underscore the growing maturity and versatility of FPGA-based acceleration for large-scale AI workloads.

Despite these advances, **scalable and latency-optimal inference for LLMs** on modern FPGAs—particularly those equipped with HBM and multi-die interconnects—remains constrained by three persistent bottlenecks:

Memory–compute affinity. The effective utilization of HBM bandwidth is often curtailed by channel imbalance, irregular access patterns, short bursts, and excessive intermediate tensor spills. In LLM decoding, *KV-cache lookups* exacerbate this problem with highly irregular and read-amplified memory accesses, leaving compute units underutilized unless the design tightly couples dataflow and memory placement.

Cross-die placement and routing limitations. When compute tiles or interconnects span multiple SLRs (super logic regions), routing congestion, clock-domain boundary issues, and timing-closure difficulties degrade achievable frequency ($F_{\max}$). Naive scaling of monolithic architectures across SLRs typically leads to sublinear throughput scaling; robust designs require *SLR-localized compute clusters* and lightweight, well-structured inter-SLR communication protocols.

Operator-shape diversity vs. fixed compute arrays. Transformer workloads exhibit significant variability in (M, N, K) dimensions across prefill and decode phases. Fixed-shape systolic arrays suffer from fragmentation, pipeline stalls, and suboptimal utilization under such variability—especially at small *batch size* . Without dynamic adaptation in array topology, precision, or folding ratio, utilization drops

2837-0449/25/$31.00 ©2025 IEEE
DOI 10.1109/ICFPT67023.2025.00048

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on April 07,2026 at 16:55:12 UTC from IEEE Xplore. Restrictions apply.

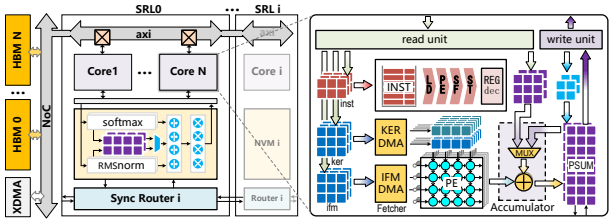

Fig. 1. Architecture of FlightOPU. The device is partitioned by SLRs; within each region, HBM channels are assigned to multiple compute cores. On-chip Sync Routers handle communication between cores across regions.

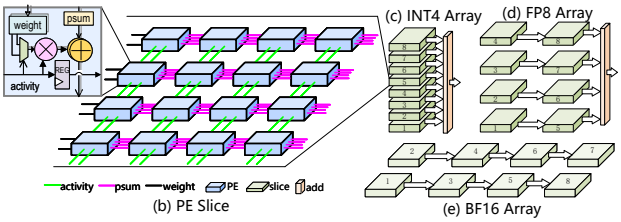

Fig. 2. Reconfigurable PE array. (a) PE unit with DSP packing. (b) A PE slice composed of 16 PE units. (c) INT4 array configuration. (d) FP8 array configuration. (e) BF16 array configuration.

and off-chip communication overhead rises.

To overcome these challenges, we present **FlightOPU**, a reconfigurable FPGA overlay processor designed for *tightly coupled compute–memory interaction*, *scalable multi-core parallelism*, and *adaptive processing-element (PE) arrays*. FlightOPU introduces three key innovations:

- **HBM-aware DSP binding:** compute clusters are physically and logically aligned with specific HBM channel groups, ensuring balanced bandwidth utilization and mitigating inter-channel conflicts;
- **SLR- and channel-aware multi-core mapping:** a compiler-driven partitioner decomposes the model graph into placement-local tiles, while a lightweight runtime synchronization fabric coordinates cross-SLR execution;
- **Reconfigurable PE arrays:** each tile dynamically adjusts its systolic array shape (e.g., $16 \times 16 \leftrightarrow 8 \times 32$) and precision mode (FP16/FP8/INT4) per operator to sustain high utilization under heterogeneous operator workloads.

Validated on a Xilinx V80 FPGA across multiple LLMs (LLaMA-7B, Qwen-7B, DeepSeek-7B, and Mistral-7B), FlightOPU achieves up to **2.5× throughput speedup** and **5.6× energy efficiency improvement**, sustaining high HBM utilization and near-linear scaling with tile count.

II. Architecture

A. Overview

Fig. 1 presents an overview of **FlightOPU**, designed for modern multi-die FPGAs with HBM. The architecture integrates three complementary strategies to jointly exploit bandwidth and compute resources. First, *SLR-aware partitioning* distributes computation and memory access within each SLR to maintain local balance, with lightweight on-chip routers minimizing inter-SLR synchronization traffic. Second, *HBM channel-bound multi-core mapping* aligns each compute core with a dedicated HBM channel, ensuring localized data access and high sustained bandwidth while selectively sharing NVM modules for auxiliary functions. Finally, each core hosts a complete *intra-core overlay processor* that integrates the dataflow engine, instruction buffer, I/O interface, and reconfigurable PE array, enabling independent operator execution.

A host-side compiler partitions the model graph and generates per-core instruction and weight files, which are transferred via PCIe to off-chip memory. During inference, the host issues start signals with offsets and token IDs; upon completion, the FPGA raises an interrupt, and the host retrieves the generated token IDs for post-processing and tokenization, supporting real-time interactive LLM inference.

B. Single-core Micro-Architecture

The single-core micro-architecture of FlightOPU, shown in Fig. 1, integrates dedicated logic and buffer structures to support a wide range of LLM operators. Each core integrates dedicated logic and buffer structures to support a wide range of LLM operators. The *memory load* unit retrieves instructions, weights, and input feature maps from off-chip memory, along with biases and hyper-parameters such as layernorm coefficients (β, γ) and scaling factors. A compact *instruction unit* decodes 32-bit commands into control signals that coordinate the core's load, compute, store, and post-processing stages. The *data fetcher* reorganizes and replicates kernel and feature-map data as required by each operator before streaming them into the *PE array*, which performs general matrix–multiply operations with reconfigurable bit-widths and DSP packing for high arithmetic utilization. Results are then merged by an *output accumulator*, which aggregates multi-round partial sums into the PSUM buffer. Finally, *special function units* implement nonlinear and model-specific operations such as softmax, layer normalization, and rotary positional encoding (RoPE), directly interfacing with the PSUM buffer to complete each operator's computation.

C. Reconfigurable PE Array

To support mixed-precision execution on FPGA DSP blocks (DSP48/DSP58), our PE array uses three complementary techniques: packed-DSP arithmetic, a modular PE-slice, and precision-adaptive array composition.

a) Packed-DSP arithmetic.: **Motivation:** maximize arithmetic throughput per DSP by executing multiple low-precision multiplies inside a single DSP block. We use two packing modes optimized for a unified BF16 activation path:

- **Pairwise BF16 packing.** Two BF16 weight lanes share a common BF16 activation so that a single DSP produces two products:

$$P_1 = W_1 \times X, \qquad P_2 = W_2 \times X.$$

- **4-way BF16 × (FP8/INT4) packing.** Two adjacent BF16 activations combine with two parallel quantized weight lanes (FP8 or INT4) to realize four multiplies in one DSP:

$$P_1 = A_1 B_1,\ P_2 = A_1 B_2,\ P_3 = A_2 B_1,\ P_4 = A_2 B_2.$$

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on April 07,2026 at 16:55:12 UTC from IEEE Xplore. Restrictions apply.

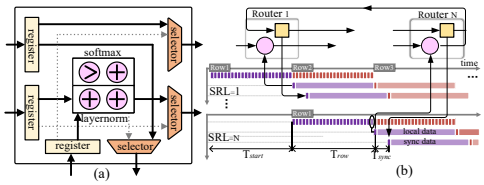

Fig. 3. On-chip synchronization router used to synchronize only the necessary values. (a) Router micro-structure showing registers and simple operations. (b) Pipeline-mode synchronization across multiple SLR regions.

These modes increase DSP utilization while keeping the activation format (BF16) consistent across the input path.

b) PE slice (4×4).: **Motivation:** reduce long-range routing and configuration overhead by building the array from a compact, repeatable tile. Each *slice* contains a 4×4 cluster of PE units; a PE unit implements a local MAC, small input/output buffers, and the logic needed for DSP packing and local accumulation. Using identical slices simplifies interconnect wiring, lowers place-and-route complexity, and makes reconfiguration for different precisions inexpensive.

c) Precision-adaptive array composition.: **Motivation:** fully utilize all DSPs and balance compute vs. memory bandwidth for each numeric mode. Full PE arrays are formed by tiling 4×4 slices into shapes chosen for the active precision. In INT4 modes we tile more slices and exploit 4-way packing to maximize parallelism; in BF16 modes the array favors wider datapaths and fewer tiles per row to match DSP and bandwidth constraints. Partial sums from tiles are merged via local adder trees into PSUM buffers. A single, common control/instruction format lets the compiler select tiling and packing strategies.

D. On-chip Synchronization Router

Motivation. In multi-core execution, global synchronization often becomes a major bottleneck when full feature vectors are exchanged across cores or SLRs. To mitigate this overhead, FlightOPU synchronizes only the minimal global statistics (e.g., the sum or maximum for layer normalization or softmax) rather than transferring entire row vectors. The proposed synchronization router, shown in Fig. 3, enables this lightweight communication by performing on-chip aggregation and selective data propagation.

As shown in Fig. 3(a), the router consists of a small register file, a reduction/compare unit (for add or max operations), and a simple handshake controller. Local accumulation is performed within each core, and only compact packets containing aggregated values are exchanged across SLRs. The router is embedded directly into the compute pipeline so that synchronization proceeds concurrently with normal computation, as in Fig. 3(b). The total latency is approximated by

$$T_{\text{total}} = T_{\text{start}} + T_{\text{row}} + T_{\text{sync}}, \tag{1}$$

where T_{start} represents the dispatch/setup delay, T_{row} the per-row computation time, and T_{sync} the additional delay from scalar synchronization and on-chip propagation. Only compact aggregated values are transmitted, inter-SLR communication remains minimal, cross-SLR cooperation practical without the heavy bandwidth cost of full-vector transfers.

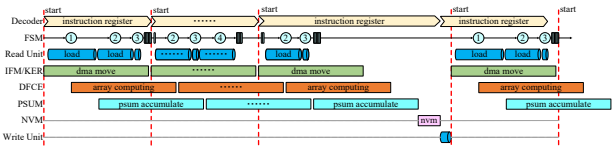

Fig. 4. Pipeline mapping: a unified load–compute–accumulate–store flow.

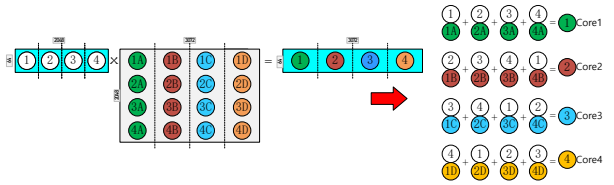

Fig. 5. Multi-core partitioning: By column-wise direction, the weights are fixed in the same HBM channel, while the activated data is interleaved to avoid conflicts.

III. Compiler and Mapping Framework

A. Pipeline mapping

As shown in Fig. 4, the compiler extracts the model into basic operators: GEMM kernels mapped to the PE array, while embedding, RoPE, softmax, and layernorm are executed on SFUs or served from NVM. Each operator follows a unified four-stage pipeline—*Load, Compute, Accumulate, and Store*—consistent with matrix-multiplication style execution.

In this flow, the compiler first decomposes the model into primitive operators, assigning GEMM fragments to the PE array and nonlinear or scalar functions to SFUs. The *Load* stage fetches instructions, weights, and activations from off-chip memory; *Compute* performs MAC operations on the PE array, with SFUs operating in parallel; *Accumulate* merges intermediate results through the adder tree or output accumulator, applying scaling or normalization when required; and *Store* writes results back to memory for subsequent layers.

This unified pipeline simplifies instruction generation and scheduling, eases cross-core or cross-SLR partitioning, and enables pipelined parallelism that improves throughput while minimizing synchronization overhead.

B. Multi core partitioning compilation Flow

As shown in Fig. 5, the overall compilation process proceeds in three stages: first, the model is parsed from frameworks like PyTorch or TensorFlow into an intermediate representation (IR) that encodes operator dependencies and tensor layouts. Next, the IR undergoes graph-level optimizations such as fusion, reordering, and tiling to minimize memory traffic and improve data locality. Finally, the optimized IR is mapped onto OPUs with balanced HBM allocation, and corresponding bitstreams and runtime control instructions are generated.

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on April 07,2026 at 16:55:12 UTC from IEEE Xplore. Restrictions apply.

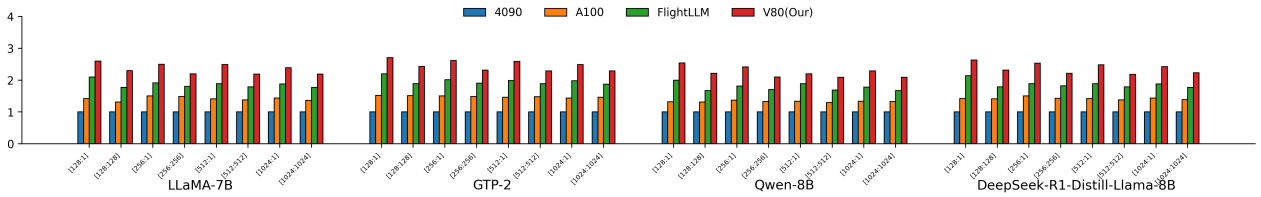

Fig. 6. The end-to-end latency varies with different prompts and generation lengths

TABLE I
FLIGHTOPU RESOURCE BREAKDOWN ON V80 FPGA

Components	LUT(k)	FF(k)	DSP	BRAM	URAM
PE Array	1478(57.4%)	3129(60.9%)	8192(75.5%)	0	0
IFM+KER	89(4.3%)	121(2.4%)	0	1910(45.2%)	148(7.7%)
PSUM	257(10.0%)	543(10.5%)	0	0	825(42.9%)
NVM	78(3.0%)	124(2.4%)	256(2.4%)	549(13.0%)	0
Router	0.5(0%)	1.9(0%)	0	0	0
Other	153(5.9%)	214(4.2%)	6(0%)	287(6.8%)	79(4.1%)
Total	2055.5(79.9%)	4132.9(80.3%)	8454(77.9%)	2746(65.0%)	1052(54.7%)

C. Operator Fusion and Scheduling

Operator fusion plays a central role in reducing intermediate memory access. For example, a matrix multiplication followed by a normalization can be fused into a single OPU pipeline, eliminating the need to write intermediate results back to HBM. The compiler automatically identifies fusion opportunities and schedules fused kernels to OPUs that can sustain the required throughput.

IV. EXPERIMENT AND EVALUATION

A. Experimental Setup

FlightOPU is implemented in Verilog HDL, synthesized and implemented on the Xilinx Alveo V80 using Vivado 2024.2, operating at 300 MHz. We developed a custom compiler in C++ to generate instructions and map models to the scalable multi-core system. For comparison, we benchmark against two high-end GPUs: NVIDIA A100 and RTX 4090, measuring GPU power consumption using the nvidia-smi tool. We use four large language models: LLaMA-7B, GPT-2, Qwen-8B, and DeepSeek-R1-Distill-Llama-8B.

B. Resource Utilization

The resource utilization of FlightOPU is shown in Table I, with usage rates of 79.9% for LUTs, 80.3% for FFs, 77.9% for DSPs, 65.0% for BRAM, and 54.7% for URAM. The PE Array, as the main computation unit, consumes 57.4% of LUTs, 60.9% of FFs, and 75.5% of DSPs. The PSUM utilizes 10.0% of LUTs and 42.9% of URAMs for accumulation operations. The IFM+KER consumes 4.3% of LUTs and 45.2% of BRAM for storage, while the NVM uses 3.0% of LUTs for nonlinear operations.

C. End-to-End Performance

We compared the performance of FlightOPU against GPUs and other works. As shown in Fig. 6, we present the end-to-end latency of four models under different input and output sequence lengths. Two key observations can be drawn: (1) When the output sequence length is 1, representing the prefill phase, FlightOPU demonstrates significantly improved speed compared to other platforms. FlightOPU is 2.6× faster than RTX 4090, 1.8× faster than A100, and 1.4× faster than FlightLLM. (2) When the input and output sequence lengths are equal, FlightOPU is 2.3× faster than RTX 4090, 1.75× faster than A100, and 1.29× faster than FlightLLM.

V. CONCLUSION

We presented a scalable OPU architecture on the V80 FPGA for efficient LLM inference with HBM. The compiler framework maps diverse operators through fusion, memory-aware scheduling, and scalable partitioning. The microarchitecture organizes PEs into OPU tiles coupled with dedicated HBM channels, achieving balanced bandwidth and flexible scalability. Experiments demonstrate near-linear throughput scaling, high HBM utilization, and consistent efficiency.

REFERENCES

[1] Q. Dong, X. Song, and Y. Chen, "An Efficient Swin Transformer Accelerator Based on FPGA," in *Proc. ASP-DAC*, 2024, pp. 1–6. doi: 10.1109/ASP-DAC58780.2024.10473931.
[2] D. Parikh, V. Ma, and P. Whatmough, "Accelerating ViT Inference on FPGA through Static and Dynamic Pruning," arXiv:2403.14047, 2024.
[3] Y. Wu, M. Song, J. Zhao, Y. Gao, J. Li, and H. K.-H. So, "TATAA: Programmable Mixed-Precision Transformer Acceleration with a Transformable Arithmetic Architecture," *ACM Trans. Embedded Comput. Syst.*, 2025. doi: 10.1145/3714416.
[4] H. Chen, Z. Zhang, and Y. Xu, "Programmable Transformer Encoder Acceleration on FPGA (ProTEA)," in *Proc. SC Workshops (H2RC)*, 2024, pp. 1–10. doi: 10.1109/SCW63240.2024.00074.
[5] V. Agostinelli, G. Ottaviano, and A. Lodi, "UltraFormer: An Efficient Transformer for FPGAs," in *Proc. FCCM*, 2025, pp. 274–284.
[6] Y. Qin, X. Zhang, and L. Liu, "Enhancing Long-Sequence Input Processing in FPGA Transformer Accelerators," in *Proc. DAC*, 2024, pp. 1–6.
[7] Y. Gao, M. Hou, and J. Wang, "The Feasibility of Implementing Large-Scale Transformers on Multiple FPGAs," arXiv:2404.16158, 2024. Multi-FPGA feasibility for large Transformers. :contentReference[oaicite:15]index=15
[8] Z. D. Zheng and X. Luo, "Achieving Low-Latency Acceleration on Multi-FPGA GPT with Communication Optimization," in *Proc. FCCM*, 2024, PhD Forum, pp. 231–233.
[9] H.-W. Oh, S. Kim, and J. Lee, "A Multimodal AI Acceleration with Dynamic Pruning and Configurable Compute on FPGA," in *Proc. FCCM*, 2025, pp. 293–302.
[10] M. Zhang, J. Wu, and Y. Bai, "FAS-Trans: Fully Exploiting FFN and Attention Sparsity for Transformer Acceleration on FPGAs," in *Proc. ACM/SIGDA FPGA*, 2025, pp. 1–10. doi: 10.1145/3676536.3676743.
[11] Lei HE, Kun WANG, Chen WU, et al. FPGA Overlay processor for AI computing (J/OL). SCIENTIA SINICA Informationis, 2025, 55 (4): 2025-04-03. 2025-10-10. https://www.sciengine.com/doi/10.1360/SSI-2024-0351.

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on April 07,2026 at 16:55:12 UTC from IEEE Xplore. Restrictions apply.